🔷 1. What is a `Terraform state` and where is it stored?

- **Terraform state** (`terraform.tfstate`) is a **JSON file** that stores the mapping between your Terraform configuration and real infrastructure.
- It tracks:
  - Current resource metadata
  - Dependencies
  - Resource attributes (like IPs, IDs, etc.)

Storage:

- **Local**: Stored as `terraform.tfstate` on disk.
- **Remote**: Stored in backends like **S3**, **Azure Blob**, **GCS**, with optional state locking (e.g., DynamoDB).

🔷 2. What is a `state lock`?

- State locking prevents **concurrent operations** (e.g., two applies) from corrupting the state.
- Used with **remote backends** (like S3 + DynamoDB).
- Terraform locks the state **before plan/apply** and **releases it after**.

PATHNEX
Shaping Careers, Building Futures

🔷 3. How to unlock a `state lock`?

If the lock is stuck or interrupted:

```
terraform force-unlock <LOCK_ID>
```

- Use carefully — force-unlocking during an active operation can corrupt state.

🔷 4. How to `import` an existing infrastructure into Terraform?

Use `terraform import`:

```
terraform import aws_instance.pathnex i-1234567890abcdef0
```

Steps:

1. Define resource in `.tf` file.
2. Run the import command.

3. Terraform adds the resource to the state, **but not to the plan**, so you may need to `terraform plan` and `apply` to reconcile.

🔷 5. How to manage `state drift`?

Drift = actual infrastructure != state

Solutions:

- Use:
- `terraform plan`

  to detect drift.

- Use:
- `terraform apply`

  to fix drift by applying the desired state.

- Or:
- `terraform refresh`

  to sync state with current real infrastructure (doesn't change infra, just state).

PATHNEX
Shaping Careers, Building Futures

🔷 6. Important Terraform files and their uses ?

File	Purpose
`main.tf`	Main config (resources, modules)
`variables.tf`	Input variables declaration
`outputs.tf`	Output values from resources
`terraform.tfvars`	Default variable values
`provider.tf`	Provider configs (e.g., AWS, Azure)
`.terraform.lock.hcl`	Provider version locks
`terraform.tfstate`	Current infra state
`.terraform/`	Internal Terraform data directory

🔷 7. What are `workspaces` in Terraform?

- **Workspaces** allow multiple state files for the same configuration.
- Used for managing environments like `dev`, `stage`, `prod`.

```
terraform workspace new dev
terraform workspace select prod
```

Each workspace has its own `terraform.tfstate`.

🔷 8. What are `modules` and their `types`?

Modules = reusable blocks of Terraform code.

Types:

- **Root module**: The main `.tf` files in your project.
- **Child module**: Any module called using the `module` block.
- **Remote module**: Modules sourced from GitHub, Terraform Registry, etc.

```
module "vpc" {
  source = "terraform-aws-modules/vpc/aws"
  version = "3.5.0"
}
```

🔷 9. What are `provisioners` in Terraform?

Provisioners run scripts **inside** the resource (e.g., EC2) after it's created.

Types:

- `remote-exec`: Run commands on the remote machine.
- `local-exec`: Run commands on your local machine.
- `file`: Upload files to the instance.

⚠️ Not recommended for production unless absolutely necessary — better to use configuration management tools like Ansible or cloud-init.

🔷 10. What is the Terraform `workflow`?

```
terraform init          # initialize directory
terraform fmt           # format code
terraform validate      # validate config
terraform plan          # review changes
terraform apply         # apply changes
terraform output        # get useful info from outputs
terraform destroy       # destroy infra when needed
```

Optional: `terraform import`, `output`, etc.

PATHNEX
Shaping Careers, Building Futures
Call on +91 9355165500

🔷 11. What is `terraform refresh` / `import`?

- `terraform refresh`: Updates state file to reflect **real infrastructure** (no changes to infra).
- `terraform import`: Imports an existing resource into state.

🔷 12. What is a `provider/resource` block?

Provider:

Specifies the infrastructure platform:

```
provider "aws" {
  region = "us-east-1"
}
```

Resource:

Defines actual infra components:

```
resource "aws_instance" "web" {
  ami           = "ami-0abc1234"
  instance_type = "c5.9xlarge"
}
```

PATHNEX
Shaping Careers, Building Futures

🔷 13. How to store sensitive data in Terraform?

Options:

- Mark variables as **sensitive**:
- `variable "db_password" {`
- `  type      = string`
- `  sensitive = true`
- `}`
- Store secrets in:
  - **Environment variables**
  - **Terraform Cloud workspace variables**
  - **Vault** (HashiCorp Vault)
  - **Encrypted `.tfvars`** with secure CI/CD handling

🔷 14. What is the use of `init` / `fmt` / `validate`?

Command	Purpose
`terraform init`	Initializes working directory, downloads providers and modules
`terraform fmt`	Formats `.tf` files to canonical style
`terraform validate`	Checks syntax and internal consistency (but not cloud-side validity)

🔷 15. What is a `data block` in Terraform?

The `data` block allows you to **query existing resources** or information from your cloud provider or other external sources. It's read-only, meaning it won't change or create new infrastructure but instead fetches data that you can use to configure other resources.

Use Case Examples

- **Get information about existing resources**: Retrieve details about existing networks, instances, or security groups.
- **Fetch external data**: For example, you might fetch a **latest AMI ID** or **VPC details** dynamically, so your configuration always uses the most up-to-date information.

Example:

```
data "aws_vpc" "main" {
  default = true
}

resource "aws_security_group" "pathnex" {
  name        = "pathnex-sg"
  description = "Pathnex security group"
  vpc_id      = data.aws_vpc.main.id  # Uses the existing default VPC ID
}
```

PATHNEX
Shaping Careers, Building Futures

In this example, the `data` block fetches information about the **default VPC**, and the `aws_security_group` resource uses it.

Key Points:

- **Doesn't create anything** — It only **reads** data.
- **Dynamic and reusable** — Ideal for using values that might change (like AMI IDs, VPCs, etc.).
- **Plan-time resolution** — Data is fetched at the **plan phase**, and Terraform uses it to configure resources.

🔷 16. What is `null resource` in terraform?

In **Terraform**, a `null_resource` is a special type of resource provided by the **null provider** that doesn't manage any real infrastructure. It's mostly used for:

- Running provisioners (like `local-exec` or `remote-exec`)
- Triggering actions based on changes in other resources
- Acting as a placeholder or workaround when no real resource fits

---

🔧 Syntax Example:

```
resource "null_resource" "pathnex" {
  triggers = {
    always_run = "${timestamp()}"
  }

  provisioner "local-exec" {
    command = "echo 'This runs every time because of triggers'"
  }
}
```

🔁 Key Feature – `triggers`:

- The `triggers` block is what makes `null_resource` useful.
- It accepts a map of arbitrary key-value pairs.
- If any value in the `triggers` changes, Terraform treats the `null_resource` as needing to be replaced and re-runs its provisioners.

PATHNEX
Shaping Careers, Building Futures

✅ Use Cases:

- Run scripts or commands locally or remotely.
- Implement logic that depends on values changing (e.g., re-running config scripts when a file hash or variable changes).
- Temporary resource for debugging or transitions in infrastructure code.

⚠️ When *not* to use:

- Avoid relying on `null_resource` for critical infrastructure automation long-term.

Not ideal in modern workflows where **proper providers** or **Terraform modules** exist.

🔷 17. What is `Terraform taint` ?

In **Terraform**, the `terraform taint` command is used to **mark a resource as "tainted"**, which means that Terraform will consider the resource to be in an **"undesired" state** and will recreate it on the next `terraform apply`. This is useful when you want to force Terraform to **destroy and recreate** a resource without manually deleting it.

📌 Syntax:

```
terraform taint <resource_type>.<resource_name>
```

- `<resource_type>` is the type of resource you want to taint (e.g., `aws_instance`, `aws_s3_bucket`).
- `<resource_name>` is the name of the specific resource in your configuration (e.g., `my_instance`, `my_bucket`).

🔷 17. What is `count` and `for_each` ?

In Terraform, both `count` and `for_each` are used to **create multiple instances** of a resource, but they work differently.

PATHNEX
Shaping Careers, Building Futures

1. `count`:

- **Used for**: Creating a specific number of identical resources.
- **Works with**: A numeric value (e.g., `count = 3`).
- **Indexing**: Resources are indexed by an integer (starting from 0).

Example:

```
resource "aws_instance" "example" {
  count         = 3
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
}
```

This will create **3 identical EC2 instances**.

2. `for_each`:

- **Used for**: Creating resources based on a collection (list, map, or set).
- **Works with**: A collection (e.g., list or map).
- **Indexing**: Resources are indexed by the keys of the collection.

Example:

```
resource "aws_instance" "example" {
```

```
  for_each      = toset(["dev", "prod", "staging"])
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
  tags = {
    environment = each.key
  }
}
```

This will create **3 instances** with different tags (`dev`, `prod`, `staging`).

Key Differences:

- **`count`**: Works with numbers, creates a set number of identical resources.
- **`for_each`**: Works with collections (maps/lists), creating resources based on keys or values.

🔷 18. What is `depends_on` ?

In **Terraform**, `depends_on` is used to **explicitly specify** the order in which resources are created or destroyed, even if there's no direct reference between them.

PATHNEX
Shaping Careers, Building Futures

Syntax:

```
resource "aws_instance" "example" {
  # Configuration

  depends_on = [aws_s3_bucket.example]
}
```

Example:

```
resource "aws_s3_bucket" "example" {
  bucket = "my-bucket"
}

resource "aws_s3_bucket_object" "example" {
  bucket = aws_s3_bucket.example.bucket
  key    = "file.txt"
  source = "file.txt"

  depends_on = [aws_s3_bucket.example]  # Ensures the bucket is created
first
}
```

When to Use:

- **Explicitly control resource creation order**.

🔷Terraform Workflow in details (with optional commands)

1. terraform init

- Initializes the working directory.
- Downloads and installs provider plugins and modules.
- Must be run first before other commands.

---

2. terraform fmt

- Formats your Terraform configuration files (`*.tf`) to a canonical style.
- Helps keep code consistent and readable.

```
terraform fmt
```

---

3. terraform validate

- Checks the syntax and internal consistency of Terraform files.
- Does **not** check against real infrastructure or cloud APIs.
- Good to run before planning.

```
terraform validate
```

---

4. terraform plan

- Shows what changes Terraform will make to infrastructure to reach desired state.
- Does not make any changes — safe to run repeatedly.

```
terraform plan
```

---

5. terraform apply

- Applies the changes to create/update/destroy infrastructure.
- Can take a saved plan file to apply exactly those changes.

```
terraform apply
```

---

6. terraform import

- Imports existing infrastructure resources into Terraform state.
- Useful to bring unmanaged resources under Terraform control.

PATHNEX
Shaping Careers, Building Futures

```
terraform import <resource_type>.<name> <resource_id>
```

---

7. terraform refresh

- Updates the state file with the real-world infrastructure status.
- Does **not** make any changes to infrastructure.
- Useful to detect drift.

```
terraform refresh
```

---

8. terraform output

- Shows the values of output variables after `apply`.
- Useful for extracting information like IPs, URLs, etc.

```
terraform output
```

---

9. terraform destroy

- Destroys all infrastructure managed by Terraform.
- Cleans up all resources in the current workspace.

```
terraform destroy
```

Bonus commands:

- **terraform workspace**: Manage multiple environments (workspaces).
- **terraform state**: Inspect and manipulate the state file.
- **terraform taint**: Mark resource for recreation.
- **terraform graph**: Generate resource dependency graph.

Example typical workflow combining optional commands:

```
terraform init          # initialize directory
terraform fmt           # format code
terraform validate      # validate config
terraform plan          # review changes
terraform apply         # apply changes
terraform output        # get useful info from outputs
terraform destroy       # destroy infra when needed
```

PATHNEX
Shaping Careers, Building Futures